Janatics.Data — Technical Summary

Overview

Purpose: The `Janatics.Data` folder is a lightweight CRUD/transaction engine and data-access toolkit focused on PostgreSQL (with MySQL/SQL Server hooks). It provides: transaction processing, field-mapper driven inserts/updates, audit logging, background DAG/trigger execution, connection pooling/optimization, validation, and utilities (auto-number, type conversion).

High-Level Architecture

- API / Caller → `TransactionService`: Incoming transaction requests are handled by `TransactionService` which orchestrates inserts/updates/deletes, child records, validation, audit enqueuing, and DAG trigger scheduling.
- Data Layer: `IEnhancedDataProvider`/`IDataProvider` implementations perform DB work.
- Implementations: `Providers/EnhancedPostgreSqlProvider.cs`, `Providers/PostgreSqlProvider.cs`.
- Connection Management: `DatabaseConnectionFactory` + `EnhancedConnectionFactory` build and manage DB connections, apply pooling/read-replica logic and metrics.
- Auditing Pipeline: Audits are enqueued during a transaction, cached for after-commit flush, and processed async by a background worker that writes to `auditlog`.
- Triggers / DAGs: DAG triggers are stored in DB and executed either by direct inserts (creating child records) or by calling an external DAG API (KTAiFlow).

Key Components / Files (quick reference)

- Project & config: `KATCRUDServices.Core.csproj`, `ConfigHelper.cs`
- Connection factories: `Factories/DatabaseConnectionFactory.cs`, `Factories/EnhancedConnectionFactory.cs`
- Providers: `Providers/PostgreSqlProvider.cs`, `Providers/EnhancedPostgreSqlProvider.cs`
- Transaction engine: `Services/TransactionService.cs`
- Auditing: `Services/AuditService.cs`, `Services/AuditDiffBuilder.cs`, `Services/AuditWriter.cs`
- Queue & background workers: `Services/AuditQueue.cs`, `Services/AuditBackgroundService.cs`, `Services/BackgroundTriggerService.cs`
- Utilities: `Services/AutoNumberService.cs`, `Services/DataTypeConverter.cs`, `Services/ValidationService.cs`
- Multiplexing / scopes: `Services/ConnectionMultiplexer.cs`, `Services/OperationScope.cs`
- Field mapping: `Services/FieldMapperService.cs`, `Models/FieldMapper.cs`

Core Data Flows

Transaction processing (simplified):

- Client constructs a `TransactionRequest` and calls `TransactionService.ProcessTransactionAsync`.
- Service validates input (optional `ValidationService`), loads `FieldMapper` config, decides Insert vs Update.
- Uses `IEnhancedDataProvider` / `OperationScope` to open a DB transaction, write main record and child records (recursively), generate auto-numbers when needed, and call `IAuditService.TryAuditAsync` BEFORE commit for UPDATE/DELETE to capture pre-state.
- On commit, `AuditService.FlushAuditLogsFireAndForget` writes change logs

to `public.auditlog` on a dedicated connection.
- Background triggers (DAGs or DirectInsert) are scheduled via `BackgroundTriggerService` (fire-and-forget).

Audit pipeline:
- During tx: build `AuditJob` and cache in AsyncLocal list if inside a transaction, else `AuditQueue.TryEnqueue`.
- After commit: cached jobs are flushed and processed by `AuditBackgroundService` which uses `AuditWriter` and `AuditDiffBuilder` to compute and store JSON changelog.

DAG trigger pipeline:
- Trigger records are read from `kat_dag_triggers`.
- For `DAGS` triggers, `DagTriggerService` calls external REST API (token handling included).
- For `DirectInsert` triggers, `BackgroundTriggerService` creates child records directly using DB calls and handles duplicate-key detection.

Database expectations / tables referenced
- `public.auditlog`, `FieldMapper`, `applicationtable`, `portaluser`, `SelectOptions`, `kat_dag_triggers`, `kat_dag_trigger_logs`, `ValidationConfigs`.
- Many queries expect GUID primary keys (`id`) and JSONB fields (PostgreSQL).

Performance, Concurrency & Reliability
- Connection pooling tuned via `DatabaseConfig` (MaxPoolSize, MinPoolSize, timeouts).
- `EnhancedConnectionFactory` collects metrics, supports read-replica selection and basic load-balancing, and exposes health/analytics APIs.
- `ConnectionMultiplexer` reduces connection churn by reusing connections across operations and parallelizing work.
- Audit queue is a bounded channel (`DropWrite` when full) to avoid slowing main operations.
- Background operations (audit writes, triggers) are fire-and-forget — they won’t block transactions but risk transient loss if process crashes before queue persisted.

Notable implementation details & risks
- Some SQL is constructed via string concatenation (e.g., parts of `DagTriggerRepository.GetTriggersForTableAsync`) — potential SQL injection or correctness risks; many other queries are parameterized.
- Audit reads sometimes open a separate connection to avoid `NpgsqlOperationInProgressException` — deliberate to avoid reader conflicts.
- `AuditDiffBuilder` relies on `FieldMapper` config to limit changelog fields; missing mappers cause fallbacks to include all fields.
- Token caching in `DagTriggerService` implements semaphore + lock to avoid parallel refreshes.
- `BackgroundTriggerService` uses in-memory queue and simple duplicate prevention; process restarts can lose pending jobs.

How to run / initialize (notes)
- Startup actions (typical):
- Construct `DatabaseConfig` and create `IEnhancedConnectionFactory` / `IEnhancedDataProvider` (DI container).
- Create `TransactionService` with its dependencies (it wires `DagTriggerRepository` and `BackgroundTriggerService`).
- Register `AuditBackgroundService` as a hosted/background worker so it

consumes `AuditQueue.Reader`.
- Initialize `DagTriggerService.Initialize(configuration)` if using DAG API.
- Config values are typically stored in `appsettings.json`.

Suggested next steps (optional)
- Parameterize remaining raw SQL concatenations and make queries consistently parameterized.
- Consider a durable queue (if audit/trigger loss is unacceptable).
- Add unit/integration tests for connection multiplexing and read-replica selection.
- Document required DB schema (FieldMapper, applicationtable, auditlog, kat_dag_triggers) in README.

Generated: 29 April 2026